nycu-swt-lab2

Part A 手動分析四塊程式碼的問題

1 findLast

(a) 指出錯在哪裡，並提出修改方式

- 錯誤：i > 0 就停止，所以不會跑到 i == 0 的時候。
- 修改：應改成 `for (let i = x.length - 1; i >= 0; i--)`。

(b) 找出一個「完全不會執行到 fault」的測資

- 測資：x = 1，y = 3
- 理由：這個測試資料都測不到 fault，因為會在判斷是否為陣列時，就拋出 TypeError('The first parameter must be an array')。

(c) 找出一個「有執行到 fault，但狀態沒變錯」的測資

- 測資：x = [1, 2, 3]，y = 3
- 理由：這個測試有執行到 fault 的部分（i > 0）的 for 迴圈判斷，但是如果我們改成 i >= 0 時，一樣會在 i = 2 時就提早 return 2，所以回傳都是 2，符合預期的結果。
- Expected = 2, Actual = 2

(d) 找出一個「狀態有變錯，但最後輸出剛好是對的」的測資

- 測資：x = [1, 2, 3]，y = 4
- Expected = -1, Actual = -1

(e) 針對 (d)，描述錯誤狀態

- 第一個 error state：x = [1, 2, 3]，y = 4，i = 0，PC 指向 `return -1`，回傳值：尚未產生
- 正確程式此時應為：PC 指向 `if (x[i] === y) {`
- 過程：
  - i = 2 時，3 !== 4，所以繼續做下去
  - i = 1 時，2 !== 4，所以繼續做下去
  - i = 0 時，因為判斷式的不同，當前的判斷不會跑到陣列為 [0] 的位置，而 i >= 0 的則會多判斷一次 1 != 4，雖然輸出的結果都為 -1。

2 lastZero

(a) 指出錯在哪裡，並提出修改方式

- 錯誤：錯在他是從 i = 0 開始，但是方法期望回傳的是最後一個 idx，所以假設當出現兩個 0 在陣列時，會先輸出前面的而非後者。
- 修改：程式碼應改成 `for (let i = x.length - 1; i >= 0; i--)`。

(b) 找出一個「完全不會執行到 fault」的測資

- 測資：x = 1
- 理由：因為 input 非 array，所以會直接拋出 TypeError('Not an array')，不會經歷 for 迴圈。

(c) 找出一個「有執行到 fault，但狀態沒變錯」的測資

- 測資：x = [0]
- 理由：因為陣列的大小為 1，所以不管怎樣 i 都從 0 開始，因為 x[0] === 0，所以會直接回傳 0。
- Expected = 0, Actual = 0

(d) 找出一個「狀態有變錯，但最後輸出剛好是對的」的測資

- 測資：x = [1, 2, 0]
- Expected = 2, Actual = 2

(e) 針對 (d)，描述錯誤狀態

- 說明：有成功的找到 x[2] 是 0 並回傳 2，雖然實際與預期的回傳值仍然是相同的，但是迴圈的順序反了，所以狀態會錯誤。
- 第一個 error state：x = [1, 2, 0]，i = 0，PC 指向迴圈條件 `i < x.length`，回傳值：尚未產生
- 正確程式此時應為：i = 2（PC 指向迴圈條件 `i >= 0`）
- 過程：
  - 有 bug 的迴圈是從 i = 0 開始
    - i = 0，1 !== 0
    - i = 1，2 !== 0
    - i = 2，發現 0 === 0，回傳 2
  - 而正確的迴圈是要從 i = 2 開始
    - i = 2，第一次就發現 0 === 0，回傳 2

3 countPositive

(a) 指出錯在哪裡，並提出修改方式

- 錯誤：for 迴圈裡面，判斷是否為 Positive 寫錯了，0 不是正數。
- 修改：應改成 `if (x[i] > 0)`。

(b) 找出一個「完全不會執行到 fault」的測資

- 測資：x = 1
- 理由：這個測試資料都測不到 fault，因為會在判斷是否為陣列時，就拋出 TypeError('Not an array')。

(c) 找出一個「有執行到 fault，但狀態沒變錯」的測資

- 測資：x = [1, 2, 3]
- 理由：對於陣列的數值判斷為 >= 0 或 > 0 結果都一樣為 true，最後的回傳值與預期值也皆為 3。
- Expected = 3, Actual = 3

(d) 找出一個「狀態有變錯，但最後輸出剛好是對的」的測資

- 結論：不可能經歷狀態有變錯，但最後輸出可以是對的情況。
- 理由：x[i] = 0 被判斷要 count++，就無法再將結果倒退回去，以 [-4, 2, 0, 2] 為測資範例。
- Expected = 2, Actual = 3

(e) 針對 (d)，描述錯誤狀態

- 第一個 error state：x = [-4, 2, 0, 2]，i = 2，count = 1，PC 指向 `count++`，回傳值：尚未產生
- 正確程式此時應為：PC 指向 `i++`
- 過程：
  - x[i] >= 0 時（count = 0）
    - i = 0，因為 -4 < 0，count = 0
    - i = 1，因為 2 >= 0，count = 1
    - i = 2，因為 0 === 0，count = 2
    - i = 3，因為 2 >= 0，count = 3

        - return 3
    - x[i] > 0 時（count = 0）
        - i = 0，因為 -4 < 0，count = 0
        - i = 1，因為 2 > 0，count = 1
        - i = 2，因為 0 <= 0，count = 1
        - i = 3，因為 2 > 0，count = 2
        - return 2
- 說明：在 i = 2 的情況，兩者的 count 計算結果不同，也造成了最後的預期結果與實際不同。

4 oddOrPos

(a) 指出錯在哪裡，並提出修改方式

- 錯誤：for 迴圈裡面，判斷是否為 odd 寫錯了，沒有考慮到出現負數且奇數的情況，例如：-3、-1。
- 修改：for 迴圈內的判斷式應改成 `Math.abs(x[i] % 2) === 1`，當出現負數的奇數數字時，才能判斷是否為 odd。

```
if (Math.abs(x[i] % 2) === 1 || x[i] > 0) {
    count++;
}
```

(b) 找出一個「完全不會執行到 fault」的測資

- 測資：x = 1
- 理由：這個測試資料都測不到 fault，因為會在判斷是否為陣列時，就拋出 TypeError('Not an array')。

(c) 找出一個「有執行到 fault，但狀態沒變錯」的測資

- 測資：x = [-4, 1, 2, 3]
- 理由：雖然原始的程式碼有執行到 x[i] % 2 === 1，但是因為 -4 本身就不是奇數，所以並不會影響狀態（-4 % 2 !== 1 且 Math.abs(-4 % 2) !== 1）以及 Expected 跟 Actual 的結果。
- Expected = 3, Actual = 3

(d) 找出一個「狀態有變錯，但最後輸出剛好是對的」的測資

- 結論：不可能。
- 理由：如果 x[i] % 2 === 1 因為是負數的奇數被判斷為 false，那後面的 x[i] > 0 也必定為 false，這會造成 count 沒有加上一，後續也沒有任何判斷把缺少的再補上，因而造成輸出結果錯誤，以 [-3, -2, 0, 1, 4] 為測資範例。
- Expected = 3, Actual = 2

(e) 針對 (d)，描述錯誤狀態

- 第一個 error state：x = [-3, -2, 0, 1, 4]，i = 0，count = 0，PC 指向 `i++`，回傳值：尚未產生
- 正確程式此時應為：PC 指向 `count++`
- 說明：錯誤在 i = 0 時就出現了。
- 過程：
    - 錯誤的程式碼
        - i = 0，-3 % 2 !== 1，因此 count = 0

Part B

第一次

我是直接在 lab2/ 這份專案下跑 Claude Code，但發現到他會讀取我在 README.md 寫的 Part A 答案，因此想到要設置一個乾淨的環境生成測試。

前置作業（Prompt A）

為了不讓 Claude Code 讀到我撰寫好的 README.md Part A，所以需要先創一個空的資料夾，讓他單純透過程式碼生成每個方法的測試案例

```
newrun() {
  rm -rf ~/hw1-run && mkdir ~/hw1-run
  cp /Users/ruserxd/Desktop/master/SWT/lab2/hw1.js ~/hw1-run/
  cd ~/hw1-run && ls && claude
}
```

這樣每次只需要在終端機打上 newrun 就會重新生成一個乾淨的環境。

前置作業（Prompt A 無 JSDoc 版）

第一輪 Prompt A 沒有出現 assertion echo，因為 prompt 和 hw1.js 裡都有 JSDoc 與 `// test: ... Expected` 註解，agent 可以直接拿來當答案。 所以準備了 `assets/promptA_nodoc/` ，裡面的 hw1.js 拿掉了 JSDoc 與註解，prompt 也只貼函式本體。

```
newrun_nodoc() {
  rm -rf ~/hw1-run-nodoc && mkdir ~/hw1-run-nodoc
  cp /Users/ruserxd/Desktop/master/SWT/lab2/assets/promptA_nodoc/hw1.js ~/hw1-run-nodoc/
  cd ~/hw1-run-nodoc && ls && claude
}
```

這樣每次只需要在終端機打上 newrun_nodoc，再貼上 `prompt_<program>.txt` 的內容即可。（findLast、lastZero、oddOrPos 是用 `claude -p` ，所以 log 格式和其他的不同）

前置作業（Prompt B）

這次的指令只需要一個空的資料夾，剩下透過 prompt 告訴其規格就好。

```
newspec() {
  rm -rf ~/hw1-spec && mkdir ~/hw1-spec
  cd ~/hw1-spec && ls && claude # 確保資料夾要空的
}
```

這樣每次只需要在終端機打上 newspec 就會重新生成一個乾淨的環境。

路徑問題

- Prompt A 的 agent 把測試檔放在根目錄，所以寫成 ./hw1.js。搬進 test/ 之後，路徑要改成 ../hw1.js。
- Prompt B 的 agent 看不到實作，所以自己猜 ./findLast 這樣形式的路徑。

```
cd /Users/ruserxd/Desktop/master/SWT/lab2
perl -i -pe 'if (m{require\(\x27\./hw1(\.js)?\x27\)}) {
  $o=$_; chomp $o;
  s{\./hw1(\.js)?}{../hw1.js};
  $_ = "// [EDITED by student] original: $o\n$_";
}' test/promptA_*.test.js

grep -n -B1 "require('../hw1.js')" test/promptA_*.test.js
```

```
perl -i -pe 'if (m{require\(\x27\./(findLast|lastZero|countPositive|oddOrPos)(\.js)?\x27\)}) {
  $o=$_; chomp $o;
  s{\./(findLast|lastZero|countPositive|oddOrPos)(\.js)?}{../hw1.js};
  $_ = "// [EDITED by student] original: $o\n$_";
```

```
}' test/promptB_*.test.js

grep -n -B1 "require('../hw1.js')" test/promptB_*.test.js
```

Claude Code 聊天紀錄

都存在 `assets/log/` 下：`log_promptA_*.txt`、 `log_promptA_noDoc_*.txt`、 `log_promptB_*.txt`、 `log_B3_lastZero.txt`。

測試檔與 log 對照

版本	測試檔	log
Prompt A（無 JSDoc，B1 表格採用）	`test/promptA_<program>.test.js`	`assets/log/log_promptA_noDoc_<program>.txt`
Prompt A 第一輪（含 JSDoc 與 `// test:` 註解）	`test/promptA_firstrun_<program>.test.js`	`assets/log/log_promptA_<program>.txt`
Prompt B	`test/promptB_<program>.test.js`	`assets/log/log_promptB_<program>.txt`
B3（lastZero 的 Part A 問答）	—	`assets/log/log_B3_lastZero.txt`

B1

B1 表格

```
node --test --experimental-test-coverage --test-coverage-include=hw1.js test/promptA_lastZero.test.js
```

Program	Prompt	# tests	# failing	Line %	Branch %	Fault revealed?
findLast	A	16	0 (+2 todo)	100	100	N*
findLast	B	34	2	100	100	Y
lastZero	A	7	0	100	100	N (echo)
lastZero	B	30	3	100	100	Y
countPositive	A	11	0	100	100	N (echo)
countPositive	B	25	5	100	100	Y
oddOrPos	A	12	0 (+1 todo)	100	100	N*
oddOrPos	B	28	6	100	100	Y

A = Prompt A 無 JSDoc 版（`test/promptA_*.test.js`，prompt 只貼函式本體，資料夾內 hw1.js 也拿掉 JSDoc 與 `// test:` 註解）。Prompt A 定義為 code only，JSDoc 屬於規格，所以不提供。 Line % / Branch % 為該 suite 對受測函式本身的覆蓋率（`node --test --experimental-test-coverage`，未覆蓋行僅屬於其他三個函式）。

* agent 從程式本身推論出 bug，寫出能揭露 fault 的測試，但用 `{ todo: ... }` 標記，`node --test` 不把它算進 failing（顯示為 `todo`），因此從測試結果看不到 fault。若移除 `todo`，該測試會失敗。

(echo) 測試把錯誤程式的實際輸出寫成 expected，所以全部通過，詳見 B1(a)。

B1(a)

拿掉 JSDoc 後，lastZero 與 countPositive 兩份 suite 都出現 assertion echo。

例 1：`test/promptA_lastZero.test.js`

```
// NOTE: With multiple zeros, the current implementation returns the index of
// the FIRST zero, not the last. This test documents the current behavior.
test('lastZero with multiple zeros returns the first zero index (current behavior)', () => {
    assert.strictEqual(lastZero([0, 1, 0]), 0);
    assert.strictEqual(lastZero([1, 0, 2, 0, 3]), 1);
});
```

- 依 JSDoc「Last index of 0 in x」，正確 expected 應為 `lastZero([0, 1, 0]) === 2`、 `lastZero([1, 0, 2, 0, 3]) === 3`。
- 今天會過：錯誤程式從 i = 0 往後找，遇到第一個 0 就 return，實際回傳 0 與 1，agent 把這兩個實際輸出直接寫成 expected。
- 修好後會壞：迴圈改成從尾端往前找，回傳 2 與 3，與寫死的 0、1 不符，這個測試就會失敗（已用 hw1_fixed.js 驗證：1 fail）。

例 2：`test/promptA_countPositive.test.js`

```
test('counts only the non-negative elements in a mixed array', () => {
    assert.equal(countPositive([-4, 2, 0, 2]), 3);
});
test('counts 0 (current behavior: condition is x[i] >= 0)', () => {
    assert.equal(countPositive([0]), 1);
    assert.equal(countPositive([0, 0, 0]), 3);
});
```

- 依 JSDoc「Count of positive elements」，0 不是正數，正確 expected 應為 2、0、0。
- 今天會過：`x[i] >= 0` 把 0 也算進去，所以實際輸出 3、1、3，agent 照抄。連題目給的失敗測資 `[-4, 2, 0, 2]` 都被寫成 expected = 3，測試名稱也改成「non-negative」去配合程式。
- 修好後會壞：改成 `x[i] > 0` 之後回傳 2、0、0，三個測試失敗（已驗證：3 fail）。

agent 在 log 中其實有注意到可能是 bug（「locks in what is probably a bug」），但因為 prompt 要求「Make sure all tests pass」，又沒有任何獨立於實作的 oracle，它只能以現有行為當作正確答案。

對照：第一輪 Prompt A 有 JSDoc 與 `// test: ... Expected` 註解可參考，所以 agent 知道正確答案，改用 `todo` 隱藏失敗或直接修改 hw1.js，沒有產生 echo。findLast 與 oddOrPos 在無 JSDoc 版中也從函式名稱與 JS 語意（`i > 0` 跳過 index 0、 `-3 % 2 === -1`）自行推論出 bug，同樣用 `todo` 處理。

B1(b)

兩者覆蓋率相同（皆 100% line / branch），但只有 Prompt B 揭露 fault，Prompt A 全部通過。Coverage 只保證 fault 那行被執行（Reachability），不保證狀態被感染、錯誤傳到輸出，或斷言的 expected 正確。Prompt A 的 oracle 來自程式本身，agent 把錯誤輸出寫成 expected 或用 todo 隱藏失敗；Prompt B 的 oracle 來自規格，獨立於實作，才能揭露 fault。

B2

B2(a)

Program	Test (quote the call and the assertion)	First condition not met (R / I / P / Reveal)	Justification
findLast	A： `assert.strictEqual(findLast([1, 2, 3], 4), -1)`（`test/promptA_findLast.test.js`，"returns -1 when the value is not present"）	P	R：有跑到迴圈條件 `i > 0`。I：i = 2、1 時 3、2 皆 !== 4；到 i = 0 時錯誤程式判斷 `0 > 0` 為 false，PC 跳到 `return -1`，正確程式（`i >= 0`）此時 PC 應在 `if (x[i] === y)` 再檢查 x[0] = 1，狀態已被感染（同 Part A (d)(e)）。P 不成立：x[0] = 1 !== 4，正確程式最後也是 `return -1`，兩者回傳值都是 -1，錯誤狀態沒有傳到輸出。

Program	Test (quote the call and the assertion)	First condition not met (R / I / P / Reveal)	Justification
lastZero	A：`assert.throws(() => lastZero(null), { name: 'TypeError', message: 'Not an array' })`（`test/promptA_lastZero.test.js`）	R	fault 在迴圈的起點與方向 `for (let i = 0; i < x.length; i++)`。x = null，`Array.isArray(null)` 為 false，直接拋出 TypeError('Not an array')，迴圈的 `let i = 0` 從未執行，i 也從未被賦值，沒有回傳值，所以連 Reachability 都沒有滿足。
countPositive	B：`assert.equal(typeof countPositive([1, -1]), 'number')`（`test/promptB_countPositive.test.js`，"returns a number"）	I	R：x = [1, -1] 是陣列，進入迴圈並執行到 `if (x[i] >= 0)`。I 不成立：i = 0 時 1 >= 0 與 1 > 0 都為 true，count = 1；i = 1 時 -1 >= 0 與 -1 > 0 都為 false，count 仍為 1。陣列中沒有 0，所以每一步的 i、count 與 PC 都和正確程式相同，最後回傳 1（正確也是 1），狀態從未被感染。
oddOrPos	hand-written：`const r = oddOrPos([-3, -2, 0, 1, 4]); assert.ok(Number.isInteger(r))`（`test/mine_reveal_oddOrPos.test.js`）	Reveal	R：進入迴圈並執行 `x[i] % 2 === 1` 或 `x[i] > 0`。I：i = 0 時 x[0] = -3，`-3 % 2` 為 -1，`-1 === 1` 為 false 且 -3 > 0 也為 false，PC 跳過 `count++`，count = 0；正確程式應執行 `count++` 使 count = 1，狀態已被感染。P：後續 -2、0 不計，1、4 計入，錯誤程式回傳 r = 2，正確應為 3，錯誤值傳到回傳值。Reveal 不成立：斷言只檢查 r 是不是整數，`Number.isInteger(2)` 為 true，測試通過，沒有看到 2 與 3 的差別。

agent 沒有產生斷言沒檢查回傳值這類的測試：Prompt B 的「回傳型別」測試（`typeof oddOrPos([1, 2, 3])`、`typeof countPositive([1, -1])`）輸入裡沒有負奇數或 0，根本沒有感染狀態，停在 I。Prompt A 的 echo 斷言與 todo 測試雖然也無法達成 Reveal（見 B2(b)），但它們的斷言其實有檢查回傳值，問題在 oracle 錯誤或失敗被隱藏，不是題目所舉「斷言沒看到錯誤值」的情況。所以 oddOrPos 這一列是自己寫的測試，放在 `test/mine_reveal_oddOrPos.test.js`，執行結果為 1 pass、0 fail。

B2(b)

不滿足。Reveal 要求測試者觀察到最終狀態中「錯誤」的部分，並且把它認定為錯誤。echo 斷言 `assert.equal(countPositive([-4, 2, 0, 2]), 3)` 雖然讀到了錯誤的回傳值 3，但 expected 本身就是從錯誤程式複製來的，oracle 跟實作一起錯，所以它把錯誤值當成正確值，永遠不會失敗。斷言有看到錯誤的值，但沒有能力判斷它是錯的，fault 沒有被揭露。Reveal 需要獨立於實作的 oracle（規格、參考實作或性質）；echo 只是把 failure 寫進了測試。

B3

B3(a)

題	agent 的答案	評分	測資	Expected（依規格）	Actual（實際執行）	理由
(a)	從前面開始找，改成從尾端往前找	Correct	x = [0, 1, 0]	2	0	跟我的答案一致
(b)	x = null	Correct	x = null	TypeError: Not an array	TypeError: Not an array	非陣列，在進入迴圈前就拋出例外
(c)	x = [1]	Incorrect	x = [1]	-1	-1	見下方說明
(d)	x = [0, 1]	Correct	x = [0, 1]	0	0	錯誤程式的 i 從 0 開始，正確程式從 1 開始，最後都回傳 0
(e)	i = 0，PC 在迴圈條件	Correct	—	—	—	四項都有寫到，見 B3(b)

Expected / Actual 的驗證指令：

```
node -e "
const {lastZero}=require('./hw1.js');
for (const x of [null,[1],[0,1],[0,1,0]]) {
  try { console.log(JSON.stringify(x), '->', lastZero(x)); }
  catch (e) { console.log(JSON.stringify(x), '->', e.name + ': ' + e.message); }
}"
```

(c) 的逐步追蹤（x = [1]）

步驟	錯誤程式	正確程式
初始化	i = 0	i = 0
檢查 x[0]	1 !== 0	1 !== 0
更新	`i++` → i = 1	`i--` → i = -1
條件判斷	1 < 1 為 false	-1 >= 0 為 false
回傳	-1	-1

在迴圈對 i 的操作並不相同，因此造成兩者狀態上的不同。

B3(b)

項目	agent 有寫嗎？	對不對？
x	有	對
i	有（0，正確程式為 1）	對
回傳值	有（not set yet）	對
PC	有（迴圈條件）	對

有詳細列出 x、i、PC、return value，沒有錯誤，和 Part A 所寫的判斷是一樣的，PC 都指向迴圈的判定（i < x.length、i >= 0）。

B3(c)

追問的 prompt：

> 請針對你在(c)提出的測資，把錯誤程式和修正後的程式都逐行追蹤一遍，包含迴圈的更新步驟。每執行完一個敘述，就比較兩邊所有變數的值。兩邊的狀態有沒有在任何時候出現差異？

結果： agent 一開始沒有發現到所給的測資並沒有符合 (c) 的題目要求：兩者的 i 更新的不同，一邊是 i++，另一邊是 i--。經過再一次的 prompt，他有發現到 x = [1] 時有出現錯誤狀態，後來有提出要使用與我在 Part A 寫的 x = [0] 當測資，才不會有錯誤狀態的出現，新的測資就有符合 (c) 的要求。

- 輸入：x = [0]
- 預期輸出：0
- 實際輸出：0

B4

- Metamorphic relation（oddOrPos）
  - 依據 test/mine_B4.test.js 進行測試：

```
const { test } = require('node:test');
const assert = require('node:assert/strict');
const { oddOrPos } = require('../hw1.js');

const cases = [
  { x: [], v: -1 },
  { x: [2, 4, 6], v: -3 },
  { x: [1, 3, 5], v: 4},
  { x: [0, 5, 6], v: 0}
];

for (const { x, v } of cases) {
  test(`append ${v} to ${JSON.stringify(x)}`, () => {
    const before = oddOrPos(x);
    const after = oddOrPos([...x, v]);
    const delta = (Math.abs(v % 2) === 1) || v > 0;

    assert.equal(after, before + delta);
  });
}
```

  - 如果 v 是一個奇數或是正數，則 oddOrPos([...x, v]) 應該等於 oddOrPos(x) + 1，否則應該等於 oddOrPos(x)。
  - 結果是 pass 兩個、fail 兩個，fail 的測資分別為：
    - Expected 是指 before + delta，Actual 則是 after
    - x: []，v: -1，Expected: 1，Actual: 0
    - x: [2, 4, 6]，v: -3，Expected: 4，Actual: 3
  - `assets/B4_failing_output.txt`：錯誤版本無法通過所有資料。
  - 把 `const { oddOrPos } = require('../assets/hw1_fixed.js');` 改成這樣，正確版本的可以通過全部的測試資料。

```
➜lab2 git:(main) ✗ node --test test/mine_B4.test.js
✔ append -1 to [] (0.360709ms)
✔ append -3 to [2,4,6] (0.056125ms)
✔ append 4 to [1,3,5] (0.04075ms)
✔ append 0 to [0,5,6] (0.034ms)
ℹ tests 4
ℹ suites 0
ℹ pass 4
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 35.061041
```

B5

1. RIPR 前三項，agent 都能很好的達到，但是最後一項的 Reveal，會因為擁有的資訊而測錯了，例如 Prompt A 只看到程式碼，因此生成出的 Expected 值有可能就會遵循著錯誤的程式碼生成，又或是因為使用者所下的 prompt，例如「Make sure all tests pass」，而造成 agents 寫的是 todo。echo 是 oracle 錯了，todo 是 oracle 對了，但失敗被隱藏，兩者都卡在 Reveal 無法達成。

2. 兩個獨立的 expected behavior 來源：

   - Specification：像 Prompt B 那樣只給 agent JSDoc，四個程式都抓到了 fault。

- Metamorphic relation：這是在 B4 上有使用的。例如：oddOrPos(x) = c，那在陣列 x 後面如果加上一個奇數或正數，預期值要是 c + 1，否則不變。

Bonus (optional): mutation testing

1. 透過 assets/use_fixed.js，將測試檔裡寫的 `require('../hw1.js')` ，在執行時換成 `assets/hw1_fixed.js` 。
2. 在有 bug 的 hw1.js 上，Prompt B 有 2 個測試會失敗，Stryker 的 initial test run 無法通過。
3. 因此先在修正版上跑 findLast 的兩份測試，確認都是 fail 0（Prompt A 另有 todo 2），Stryker 的 initial test run 才能通過。 `NODE_OPTIONS="--require $PWD/assets/use_fixed.js" node --test test/promptA_findLast.test.js`
4. 透過撰寫 stryker.A.json、stryker.B.json 兩個設定檔執行 Stryker（自動製造 mutant、自動跑測試、自動計分）。
5. 結果：
   - 存放在 `assets/stryker_prompt*_findLast.txt`
   - 兩個分數：Prompt A 91.30%、Prompt B 86.96%
   - 有一個變異體不管在 A 或 B 都存活下來，把 for 的起始值改成 `let i = x.length + 1` ，但這被稱為 equivalent mutant（等價變異），不管怎麼測試都不可能成功殺死，所以實際上分數可以更高。
   - 有一個變異未能在 A 殺死，但 B 可以， `for (let i = x.length - 1; i > 0; i--)` ，修改了 i >= 0 -> i > 0，也就是作業本來錯誤的部分。
   - 解釋： 變異與正常版本針對 x = [2, 3, 5]、y = 2 各執行一次的結果：

```
➜  lab2 git:(main) ✗ node -e "
const m = require('./hw1.js');               // mutant(i > 0)
const f = require('./assets/hw1_fixed.js');  // 正確版本(i >= 0)
console.log('mutant :', m.findLast([2, 3, 5], 2));
console.log('fixed  :', f.findLast([2, 3, 5], 2));
"
mutant : -1
fixed  : 0
```

     - R：這筆測資皆會進入迴圈，都有執行到被改動的條件。
     - I：
       - 變異版本： `0 > 0` 為 false，PC 指向 `return -1`
       - 正確版本： `0 >= 0` 為 true，PC 指向 `if (x[i] === y)`
       - 正常的版本：
         - i = 2，5 !== 2
         - i = 1，3 !== 2
         - i = 0，2 === 2
         - 回傳 0
       - 變異的版本：
         - i = 2，5 !== 2
         - i = 1，3 !== 2
         - 跳出迴圈
         - 回傳 -1
     - P：兩個版本的回傳值不同，代表錯誤有傳到輸出。
     - Reveal： Prompt A 中 y 在 index 0 的測試有 2 個，但都被標成 `todo` 。 將 todo 拿掉後重新執行（複製一份暫時的測試檔，不修改原檔）：

```
== 有 todo(原檔)，在 mutant 上
ℹ pass 14
ℹ fail 0
ℹ todo 2
== 拿掉 todo，在 mutant 上
ℹ pass 14
ℹ fail 2
ℹ todo 0
```

assertion 其實有測試到變異產生的錯誤，但因為測試案例被標成 todo，造成 mutant 存活，Reveal 無法成立。

- 結論：雖然 Prompt A 的分數比 B 高，但它卻殺不掉最關鍵的那個 mutant（i >= 0 -> i > 0），也就是作業原本的 fault；B 沒殺掉的只是錯誤訊息被清空這類問題。所以 mutation score 跟 coverage 一樣，不能單憑總分的高低，去斷定測試的品質是好或壞。

AI 使用

使用的工具

- Claude Code，model：Opus 5.5
  - version：2.1.289（Claude Code）
  - 日期：2026/10/1–2026/10/4
  - 聊天紀錄都存在 assets/log 內
- claude.ai（聊天介面），日期 2026/10/3–10/4
  - https://claude.ai/share/05f61ecc-56ac-492e-bfa1-f367a1dcd7a8
- 用途：md 檔案的檢查與提示、報告修改建議、作業的設定與驗證方法